Market Breadth Indicator: Application Walkthrough

Oct 12, 2026 · @Abhirag Kulkarni

Overview

The app measures the health of the Indian market at end of day from the top ~1000 NSE stocks. It reports two breadth indicators for each market-cap bucket and turns them into one of six market situations.

The two indicators

- **Volume Ratio (VR)**: the number of stocks trading at 1.5× or more of their 20-session average volume, divided by the number trading below 0.5× of it. Above 1.0 means demand; below 0.5 means a quiet, choppy market.
- **% above EMA (P)**: the share of stocks whose split-adjusted close is above its 10-, 20- or 50-day exponential moving average.

The six buckets. Each indicator is counted separately per bucket and for ALL. ALL is counted from the stocks directly, never averaged from the buckets.

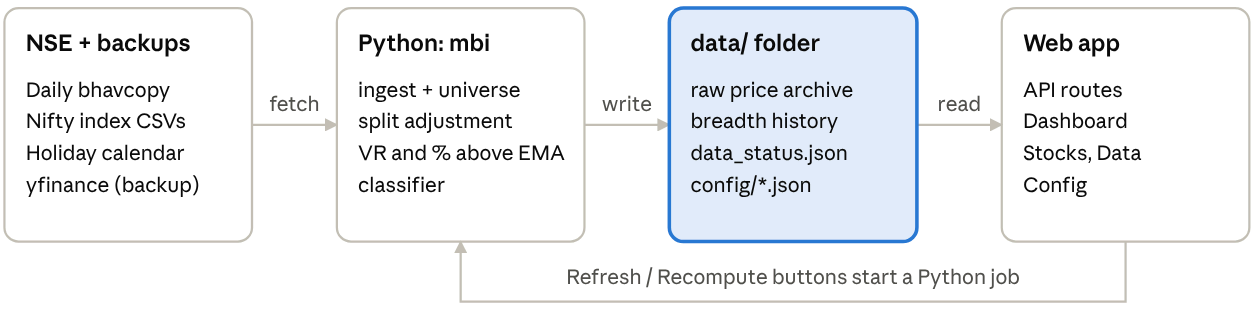
Bucket	Market-cap rank	Stocks	Where the list comes from
Largecap	1–100	100	Nifty 100 index file
Midcap	101–250	150	Nifty Midcap 150 index file
Smallcap	251–500	250	Nifty Smallcap 250 index file
Microcap	501–750	250 (251 today)	Nifty Microcap 250 index file
Nanocap	751–1000	250	Ranked by the app from the bhavcopy archive
ALL	1–1000	all of the above	Union of the five buckets

The six market situations. Each day and bucket gets one state, shown everywhere with a fixed colour, an icon and a two-letter code so colour is never the only cue.

Code	State	Meaning	Playbook (what to focus on)
EU ✦	Euphoria / Overheated Market	Highly extended market; breakouts may fail	Parabolic shorts, gap-up shorts, playing the index trend
BD ▲	Bullish Domination	Stocks move upward easily	Breakouts, gaps, strong-structure reversal setups
LD ◇	Lack of Demand	No strong demand; stocks fail to follow through	Intraday breakouts, shorting gaps, reversals and pullbacks
LS ◆	Lack of Supply	Sellers pausing after a strong decline	Resilient stocks that are not falling further; breakouts from strong structures
BR ▼	Bearish Domination	Majority of stocks are falling	Short setups, stocks entering stage 4
PA ✖	Panic / Overheated Decline	Oversold, bottoming situation	Reversals; parabolic long setups in strong-structure or declining stocks

Two halves. A Python package (`mbi`) downloads and archives prices, computes the indicators and writes plain files under `data/`. A Next.js web app reads those files and draws the dashboard; its Refresh and Recompute buttons simply start the Python jobs. There is no database server.

Python jobs write files; the web app reads them and starts jobs



how the app fits together · 4 parts

The data/ folder is the only link between the halves: if the web app is closed, the Python jobs still run, and the app shows whatever they last wrote.

Getting started

Four commands take you from an empty folder to a working dashboard; after that, one command a day keeps it current.

One-time setup (run from the project folder `d:\Abhirag code\Abhirag MBI`):

1. Install the Python packages: `pip install -r requirements.txt` (pandas, numpy, pyarrow, requests, yfinance, pytest).
2. Load history: `python -m mbi backfill --from 2025-01-01`. This builds the universe, downloads one bhavcopy per trading day (about 4 minutes for 440 sessions), ranks the nanocap tier (about 6 minutes of share-count lookups the first time) and computes every indicator. Go further back for longer history; the archive keeps everything.
3. Build the web app once: `cd web`, then `npm install` and `npm run build`.
4. Start it: `npm start` (inside `web`), then open http://localhost:3000.

Every trading day, after about 6:30 PM IST (when NSE publishes the bhavcopy), either press **Refresh data** in the app header or run `python -m mbi daily`. The job fetches any sessions missed since the last run, retries recent failed days, recomputes and reclassifies. It takes a few seconds.

Twice a year, after the end-of-June and end-of-December index reconstitutions, the daily job refreshes the universe automatically. You can force it any time with `python -m mbi universe --refresh`.

Trying the app without network access. `python -m mbi demo` writes about 360 sessions of synthetic data so you can explore every page. Point it at a separate folder first (set the environment variable `MBI_DATA_DIR`, for example to `C:\temp\mbi-demo`, and start the web app with the same variable). The demo refuses to run on a folder that holds real data, and the status log labels its days as synthetic.

Optional extras

- Browser fallback for NSE: `pip install playwright` then `python -m playwright install chromium`. Without it, the app still works; the fallback step is skipped and logged.
- AMFI market-cap file for the nanocap ranking: `pip install openpyxl` and pass the file with `--amfi-file`.

The header and global controls

The header stays at the top of every page and carries the controls that apply everywhere. Reading left to right:

Element	What it shows or does
Market Breadth · India	Returns to the dashboard, keeping the chosen EMA.
Navigation	Dashboard, Stocks, Data & Quality, Config. The current page is highlighted.
Session	The latest trading date in the breadth history.
Universe	The version (build date) of the stock list in use. Hidden on narrow screens.
Refreshed	When the last job finished. Hidden on narrow screens.
Status badge	The health of the data, in one word plus an icon (table below). Click it to open Data & Quality.
EMA 10 / EMA 20 / EMA 50	The global EMA selector, default 20.
Refresh data	Starts `python -m mbi daily` in the background. Shows “Refreshing…” and is disabled while a job runs.
☾ / ☀	Switches between light and dark themes.

Status badge levels. The badge shows the worst condition present.

Icon	Colour	Typical label	Meaning
●	Green	Data OK	The latest day loaded cleanly from the NSE bhavcopy.
▲	Yellow	Recovered, Fallback source, Universe stale, No data yet	NSE needed a retry, a day came from a backup source, or the nanocap list is stale.

Icon	Colour	Typical label	Meaning
◐	Orange	Partial	Some symbols are missing for the latest day and were left out of the counts.
■	Red	Job failed, Offline / stale, Failed	The last job crashed, NSE was unreachable, or the latest day failed to load.

The EMA selector decides which EMA length drives the % above EMA highlight, the heatmap, the market-situation card, the timeline, the state map, the statistics and the KPI tiles. All three lengths are always computed and stored, so switching never changes the stored history. The choice is saved in the address bar (`?ema=20`), so a copied link opens on the same setting, and in the browser, so it is remembered next time.

The job strip. While a job runs, a thin strip under the header shows its name and the last line of its output, updated every 1.5 seconds. It stays for two minutes after the job ends: "finished", or "failed" with a **View log** link. When a job finishes, every page reloads its data automatically.

Banners. Under the header, one banner appears per problem, in plain language, with a coloured bar and icon for its level (info, warning, partial, error). A banner can have a **details** toggle (for example the list of missing symbols, or a crash traceback) and a link to the Data & Quality page. The × dismisses a banner for the rest of the browser session; it returns if the problem is still there in a new session. Section "Errors, fallbacks and what the UI shows" lists every banner.

Dashboard page (/)

The dashboard answers "what situation is the market in today, and how did it get here?". It runs top to bottom from today's verdict to history and comparisons. If no history exists yet, the page shows a single empty-state panel telling you which command to run.

1. Market situation card

The card at the top gives today's state for one bucket (ALL by default) at the selected EMA.

- **State badge**: colour, icon, code and short name, e.g. "◆ LS · No supply". The **?** button beside it opens an explanation such as "P(EMA20) = 21.3%, VR = 0.28, dP = +2.7, rule: L × low → No supply".
- **Sessions in state**: how many consecutive sessions the confirmed state has held.
- **Raw today**: shown only when today's raw classification differs from the confirmed state, i.e. a change is waiting for its second day of confirmation.
- **Four figures**: P (% above the selected EMA), VR, dP (change in P over 5 sessions) and the rule that fired, e.g. "L × low" or "Panic override".
- **Context line**: date, high- and low-volume counts, and valid stocks out of the universe (e.g. "983 valid of 1001").
- **Playbook panel**: the meaning of the state and what to focus on (the table in the Overview).
- **Show raw state (no smoothing)** checkbox: switches the card, the bucket row, the timeline and the statistics from confirmed to raw states.
- **Bucket row**: six buttons, one per bucket, each with its state chip. Clicking one makes it the selected bucket for the whole page. Hovering shows the explanation.

2. KPI tiles

Six tiles, one per bucket. Each shows:

- the latest volume ratio, its change since the previous session, and its regime label (for example "Choppy and muted"). A ratio of "–" means no stock was low-volume, so the ratio is undefined; the tile then shows the high count;
- a 30-session sparkline of the volume ratio;
- % above EMA10, EMA20 and EMA50 with day-on-day change; the selected EMA is bold and shaded;
- a 30-session sparkline of the selected EMA's %;
- valid stocks out of the bucket's universe;
- the state chip in the corner.

Clicking a tile selects that bucket for the page, exactly like the bucket row.

3. Control bar

A bar that sticks under the header while you scroll.

Control	Effect
1M / 3M / 6M / 1Y / All	Date range for every chart. Default 6M. Changing it resets the zoom.
Bucket for single-bucket charts	The bucket used by the lower panel of the VR chart, the % above EMA chart, the state map and the statistics. Same as clicking a tile.
Range read-out	The dates and number of sessions on screen, or the zoomed window.
Reset zoom	Appears only when zoomed; returns to the full range.

Zooming works the same on every time chart, and all of them follow each other. Scroll the mouse wheel over a chart or pinch, drag the handles of the slider under a chart, or use the toolbox at the chart's top right: "Brush to zoom" (drag a box), "Undo zoom", "Reset" and "PNG" (save an image). Every chart card also has a **CSV** button that downloads exactly the rows on screen.

4. Volume ratio chart

- **Upper panel**: one line per bucket plus a thicker ALL line, labelled at its end. Click names in the legend to hide or show buckets. Shaded horizontal bands mark the regimes: extreme low demand (< 0.25), choppy (0.25–0.5), improving (0.5–0.7), building demand (0.7–1.0), high demand (1.0–2.0) and extreme excess (≥ 2.0). Red tints are fear, blue tints are demand.
- **Linear / Log** switch: log is the default because a few days with very few low-volume stocks produce spikes (Largecap reached 40) that flatten everything else.
- **Lower panel**: for the selected bucket, the high-volume count as a blue bar above zero and the low-volume count as a red bar below. This shows what drove the ratio.
- **Hollow dots** mark days built from a fallback source or with symbols missing.
- **Tooltip**: for the hovered date, every bucket's ratio and "high/low of valid" counts, the ALL regime and the data source.

5. % of stocks above EMA chart

- **One bucket** view: three lines (EMA10 blue, EMA20 orange, EMA50 green) for the selected bucket on a 0–100% scale. The line for the selected EMA is thicker and labelled. Lines at 20% and 80% with shaded oversold and overbought zones.
- **Small multiples** view: six small panels, one per bucket, all on the same 0–100% scale, so buckets can be compared at a glance.
- **Tooltip**: all three percentages with "above of valid" counts, and the data quality on non-ok days.

6. Market situation timeline

A strip with one row per bucket and one coloured cell per session, aligned to the same dates as the charts above. The legend underneath names each colour with its icon and code. When you zoom to 70 sessions or fewer, each cell also shows its letter code. Hovering a cell shows the state name and the full explanation. It follows the raw/confirmed checkbox.

7. Heatmap

Days across, buckets down. A switch chooses the colouring:

- **% > EMA**: light blue (near 0%) to dark blue (near 100%).
- **Volume ratio**: red below 1.0, grey at 1.0, blue above, on a log scale so 0.25 and 4.0 are equally strong.

Hovering a cell shows both indicators with their counts.

8. State map

A scatter of the selected bucket: % above the selected EMA across, volume ratio up (log scale). The background is the decision table drawn as a grid of coloured zones, using the edges from the current config, with each zone's state code written in it. A cell marked "LD / BD*" switches state on a trend condition. The darker band at the top-left is the panic override.

- Grey dots: every earlier session.
- Dark line with growing dots: the last 10 sessions, fading into the past, so you can see the market moving between zones.
- Pulsing dot with its date: the latest session, coloured by its state.

- **Replay** slider: drag back in time to replay the trail as it stood on any date; **latest** jumps back.
- A null volume ratio (no low-volume stocks) is plotted at the top.

The bucket picker on the card changes the selected bucket for the whole page.

9. State statistics

For the selected bucket and range (and zoom), at the selected EMA:

- a table of days in each state, share of sessions (with a bar), number of runs, average run length and longest run;
- a **transition matrix**: rows are the state moved from, columns the state moved to, darker blue for more frequent moves. A **CSV** button exports it.

10. Compare days

Pick **Day A** and **Day B** from the date lists (default: five sessions ago and the latest). The table shows, for every bucket, the state change A → B and, for volume ratio, % above the selected EMA, N high, N low and n_valid, the value on A, the value on B and the difference B − A. **CSV** exports the comparison. The same comparison is available from the command line with `mbi compare`.

Stocks page (/stocks)

The Stocks page shows the per-stock inputs behind one day's counts, so you can see exactly which stocks were high-volume, which were above their EMAs and which were left out, and why.

Controls (top right of the card)

Control	Effect
Session	Any date in the history, newest first. The last 30 sessions are pre-computed; an older date is computed on demand from the archive the first time you open it (a few seconds, noted under the controls).
Bucket	All buckets, or one bucket.

Control	Effect
Show	All symbols, valid only, or excluded only (the count is in the label).
Symbol search	Filters as you type.
CSV	Downloads the rows currently shown.

Columns. Click a column header to sort; click again to reverse. The default sort is rel_vol, highest first.

Column	Meaning
Symbol	NSE symbol.
Bucket	Bucket in the universe version in force that day.
Close	Raw close. A small "÷5" beside it means a listed split adjusts this date's prices by that factor in the calculations.
Volume	Raw traded quantity, Indian digit grouping.
rel_vol	Today's split-adjusted volume ÷ the average of the previous 20 sessions. "▲ high" = counted in N high (≥ 1.5); "▼ low" = counted in N low (< 0.5).
EMA 10 · 20 · 50	"▲ above" (blue) or "▼ below" (red) the EMA for each length; the selected EMA is bold. Hover a chip for the EMA value.
Status	"valid", or "excluded:" with the reason (see below). Excluded rows are greyed.
Archive	**OHLCV** downloads that stock's full raw price history from the archive as CSV.

Why a stock is excluded from a day's counts

- **not traded**: no row in that day's bhavcopy (suspended, or a REIT/series not traded that day), or zero volume.
- **history < 50 sessions**: a recent listing; it joins the counts after 50 sessions.

- **suspected unadjusted event**: the close moved below 0.6× or above 1.6× the previous close and the move is not covered by the split list (likely a split, bonus or demerger). Also listed on Data & Quality.
- **no volume average**: fewer than 20 earlier sessions to average.

A grey note above the table appears when the chosen day itself was not clean (recovered, fallback or partial), naming the source used.

Data & Quality page (/data)

This page is where you check whether the numbers can be trusted: what the last job did, where each day came from, and which stocks look wrong. Every banner and the status badge link here.

Top row: three cards

- **Last job**: the command, start and finish times, result (● ok, ■ failed, or running/interrupted), its summary line, and on failure the error and a short traceback. Buttons: **Run daily** (same as Refresh data) and **Recompute from archive** (rebuilds every indicator offline from the stored prices; use it after editing the split list).
- **Universe & calendar**: the universe version and its age; where ranks 1–750 came from (`nse_index_csv` = fresh download, `previous_version`, `universe_history:<date>` or `seed:universe_seed_750.csv` when downloads failed); the nanocap status (ranked, stale or provisional); counts per bucket; any universe warnings (for example "MICROCAP: expected 250, got 251"); the holiday-calendar source (`nse`, `cache` or `weekday_assumption`); and whether NSE was reachable at the last refresh.
- **Downloads**: `breadth_history.csv` (the whole indicator history, opens in Excel); a symbol box with **Raw OHLCV** (the archived, unadjusted prices) and **Split-adjusted** (the same history with the `adj_*` columns and factor); and counts of days by status.

Data quality · suspected unadjusted events. Every day on which a stock's close moved below 0.6× or above 1.6× its previous close with no matching split-list entry. Columns: date, symbol, bucket, the ratio, and a ready-to-paste JSON line for the split list with the factor worked out (1 ÷ ratio). Review the event before pasting: a demerger or a genuine crash is not a split. On the current data this lists, among others, LICI, VEDL and INDIAGLYCO (the price moves look like a 1:1 bonus and two demergers; verify before adding them), and three seed splits whose ex-dates are one session early (TDPOWERSYS, KIRLPNU, E2E).

Split list validation. The result of checking `data/config/split_adjustments.json` at the last compute: a green “All rows valid” or one line per skipped row with its reason (duplicate symbol and date, invalid date, factor not above 0, symbol not in the archive), then the table of splits in use (symbol, ex-date, factor, type).

Per-day status log. One row per stored session: date (marked “stale” when the network was down at the last refresh), status with icon, source used, attempts, missing symbols (first six shown, hover for all), excluded stocks summarised by reason, and errors and notes. The **only days with issues** checkbox (on by default) hides clean days.

Recent events. The last 100 entries recorded by the NSE client and the jobs: recoveries after a 403, fallbacks, failures and universe builds, newest first, with timestamps.

Run log. The live output of a running job (last 80 lines, refreshed every 1.5 seconds), then the full log file of the last job (`data/meta/logs/…`). **Reload** re-reads the file. The “View log” link in failure banners jumps here.

Config page (/config)

The Config page edits the market-situation thresholds and shows, before you save, how many stored days would change state. It only changes the state columns; the volume ratio, the percentages and the price archive are never touched.

Header buttons

- **Reset to defaults**: loads the built-in defaults into the form (not saved yet), so you can see the preview first.
- **Save**: writes `data/config/regime_config.json`. Enabled only when something changed. The stored history keeps its old states until a reclassify runs.
- **Save & reclassify history**: saves, then runs `mbi recompute --states-only`, which re-runs the classifier over every stored day in a few seconds. The dashboard picks up the new states when the job ends.

The fields, in order

Group	Field	Default	What it controls
P zones	L from / M from / H from / VH from	20 / 40 / 60 / 80	Lower edges of the five % above EMA zones: VL < 20, L 20–40, M 40–60, H 60–80, VH ⩾ 80. A value exactly on an edge belongs to the upper zone.
VR zones	mid from / high from	0.5 / 1.0	Volume-ratio zones: low < 0.5, mid 0.5–1.0, high ⩾ 1.0.
VR zones	extreme from	2.0	Drawn on the state map as the extreme line.
Panic override	enabled, VR ⩾, and P <	on, 2.0, 30	Any day with VR at or above the first value and P below the second is Panic, before the table is read.
Euphoria override	enabled, P ⩾	on, 80	Any day with P at or above this is Euphoria, whatever the volume.
Trend & smoothing	use dP trend input	on	Enables the trend conditions in the M×mid and L×low cells. Off = dP is treated as 0.
Trend & smoothing	dP window (sessions)	5	dP = P today − P this many sessions ago.
Trend & smoothing	confirm_days	2	Sessions a new raw state must hold before the confirmed state switches. 1 = no smoothing.

Decision table (read-only grid). Shows the current table: rows VH, H, M, L, VL; columns VR low, mid, high; each cell tinted with its state colour. A cell such as “LD / BD*” is conditional: it gives the second state when its condition holds. Edit cells in the Raw JSON box below.

Live preview. Half a second after any change, the Python classifier re-runs on the stored history with the edited settings. Three cards (EMA10, EMA20, EMA50; the selected EMA is outlined) show how many day × bucket rows would change state, on how many dates, and the resulting count of rows in each state.

Raw JSON. The whole config, including the decision table and the volume-ratio regime bands used for the chart shading and KPI labels. Edit the text and press **Apply JSON** to load it into the form and the preview; an invalid JSON shows an error and changes nothing. Then Save as usual.

Configuration files reference

Four JSON files in `data/config/` hold every setting; each is created with defaults on the first run and can be edited by hand. After editing one, run the command in the last column.

File	Holds	Apply with
`split_adjustments.json`	The split and bonus list (`SPLIT_ADJUSTED_STOCKS`)	`python -m mbi recompute`
`regime_config.json`	Classifier thresholds, decision table, VR regime bands	`python -m mbi recompute --states-only` (or the Config page)
`calendar.json`	Weekend trading sessions and short sessions	`python -m mbi recompute`
`app_config.json`	Volume-average and detail-table options	`python -m mbi recompute`

split_adjustments.json

A list of entries. The app never downloads corporate actions; any split or bonus missing here distorts that stock's EMAs and volume average until it is added. The Data & Quality page suggests entries for unexplained price jumps.

```
[
  {"symbol": "KOTAKBANK", "ex_date": "2026-01-13", "old_fv": 5, "new_fv": 1,
"factor": 5.0, "type": "split"},
  {"symbol": "ABC", "ex_date": "2026-05-29", "type": "bonus", "bonus_x": 1,
"bonus_y": 1}
]
```

- `factor` = old face value ÷ new face value for a split; for a bonus of x:y it is (x + y) ÷ y. You can omit `factor` and give `old_fv` / `new_fv` or `bonus_x` / `bonus_y` instead.
- `ex_date` is the first session that trades at the new price. Every row dated before it has prices divided by the factor and volume multiplied by it.
- Rows with a duplicate symbol and date, an invalid date, a factor of 0 or less, or a symbol not in the archive are skipped and listed on Data & Quality.
- The initial content is the seven splits in `splits_seed.csv`. Three of their dates look one session early on real data (TDPOWERSYS 2026-08-24, KIRLPNU 2026-08-18, E2E 2026-06-05).

regime_config.json

Key	Default	Meaning
`p_zones`	`{"L": 20, "M": 40, "H": 60, "VH": 80}`	Lower edges of the P zones.
`vr_zones`	`{"mid": 0.5, "high": 1.0, "extreme": 2.0}`	Lower edges of the VR zones.
`table`	the 5 × 3 table in “How the numbers are calculated”	State per P zone × VR zone. A cell is a code (`"BD"`) or a conditional object (below).
`overrides.panic`	`{"enabled": true, "vr_min": 2.0, "p_max": 30}`	Panic when VR ≥ vr_min and P < p_max.
`overrides.euphoria`	`{"enabled": true, "p_min": 80}`	Euphoria when P ≥ p_min.
`null_vr_zone`	`"high"`	Zone used when VR is undefined (no low-volume stocks).
`trend`	`{"enabled": true, "dp_window": 5, "missing_dp": 0}`	dP settings; `missing_dp` is used for the first sessions, before a dP exists.
`confirm_days`	`2`	Smoothing; 1 switches it off.

Key	Default	Meaning
`vr_regimes`	six bands	Labels and edges for the VR regime names and chart shading.
`boundary_margin`	`{"p": 2, "vr": 0.05}`	Reserved for near-edge warnings.

A conditional cell, as used for M × mid by default:

```
{"state": "LD", "alt_state": "BD", "alt_when": {"vr_min": 0.7, "dp_gt": 0}}
```

`alt_when` accepts `vr_min`, `dp_gt`, `dp_lt` and `dp_gte`; all listed conditions must hold for `alt_state` to apply.

calendar.json

- `extra_sessions`: weekend dates that were full trading sessions (default 2024-01-20 and 2025-02-01, Budget Saturday). Weekends are otherwise skipped.
- `short_sessions`: dates that are archived but left out of every calculation, such as Muhurat trading (default 2021-11-04, 2022-10-24, 2023-11-12, 2024-11-01, 2025-10-21). Add the 2026 Muhurat date once it is known. Please verify these dates.

NSE holidays come from NSE's holiday calendar and are cached in `data/meta/holidays_cache.json`; a weekday that returns no bhavcopy and is not a known trading day is remembered in `data/meta/no_session_dates.json`.

app_config.json

Key	Default	Meaning
`include_today_in_avg`	`false`	`false`: the 20-session volume average uses the 20 sessions before today. `true`: it includes today.
`detail_days`	`30`	How many recent sessions get a pre-built per-stock table for the Stocks page; older days are computed on demand.

Environment variables

Variable	Default	Used by
`MBI_DATA_DIR`	`<project>/data`	Both halves: where all data lives. Set it the same for Python and the web app.
`MBI_PYTHON`	`python`	Web app: the interpreter it starts for jobs.
`MBI_ROOT`	the folder above `web`	Web app: where the `mbi` package lives.

Command-line reference

Every job runs as `python -m mbi <command>` from the project folder. Jobs that change data write a log to `data/meta/logs/` and record their result on the Data & Quality page; a crash is logged and leaves earlier data untouched.

Command	Options	What it does
`daily`	`--date YYYY-MM-DD`, `--offline`	Ingests every session since the last archived one, retries failed, fallback or partial days from the last 10 days, then computes and classifies. `--date` treats that date as the latest session; `--offline` skips the network and recomputes from the archive. Refreshes the universe automatically after each half-yearly reconstitution.
`backfill`	`--from` (required), `--to`, `--fallback`, `--rebuild`, `--no-nanocap`	Downloads a range of bhavcopies (UDiFF format from 2024-07-08, the older format before that), skipping dates already archived, then ranks the nanocap tier if it is stale and recomputes everything. `--fallback` also uses the NSE API and yfinance for missing days; `--rebuild` re-downloads and replaces archived days (the old files are kept as `.superseded-*` copies).

Command	Options	What it does
`recompute`	`--from`, `--to`, `--states-only`	Rebuilds the indicators from the archive with no network. Default: the whole archive. `--states-only` only re-runs the classifier (seconds).
`universe`	`--refresh` [`--nanocap-only`] [`--amfi-file PATH`] [`--offline`]	Builds a new universe version dated today. `--nanocap-only` keeps ranks 1–750 and re-ranks 751–1000.
`universe`	`--diff [A] [B]`	Promotions, demotions, additions and removals between two versions (default the last two; `seed` means the seed file).
`universe`	`--show [VERSION]`	Prints a version (default the latest).
`consolidate`	none	Writes all daily price files into one `data/prices/prices_all.parquet` for backtesting.
`compare`	`--date A --date B`, `--ema 10|20|50`, `--json`	Prints two days side by side per bucket, with differences.
`export-stock`	`SYMBOL`, `--adjusted`	Prints a stock's archived OHLCV as CSV; `--adjusted` adds the split-adjusted columns.
`export-history`	none	Prints the indicator history as CSV (falls back to the parquet copy).
`day-detail`	`--date`	Builds the per-stock table for one day (used by the Stocks page).
`print-config`	`--defaults`	Prints the current (or default) classifier config.
`classify-preview`	`--config PATH`	Reports how many stored states a candidate config would change (used by the Config page).
`demo`	`--days 300`, `--seed 7`, `--force`	Writes synthetic data; refuses to touch real data.

Examples

- Catch up after a week away: `python -m mbi daily`
- Load more history: `python -m mbi backfill --from 2021-01-01`
- Apply an edited split list: `python -m mbi recompute`
- See who moved between buckets at the last refresh: `python -m mbi universe --diff`
- Compare two days for EMA50: `python -m mbi compare --date 2026-10-01 --date 2026-10-09 --ema 50`

How the numbers are calculated

Everything is recomputed from the stored raw prices plus your split list, so any past date can be reproduced offline and gives the same answer whether it was computed by the daily job or by a full recompute.

Which stocks count on a day

For each date the app uses the universe version in force that day (the latest version dated on or before it; older dates use the earliest version). A stock counts (is **valid**) only if it traded that day with volume above zero, has at least 50 sessions of history in the last 320, has a 20-session volume average, and has no suspected unadjusted price jump that day. `n_excluded` = universe size − valid.

Split adjustment

For every row dated before a listed ex-date, prices are divided by the factor F and volume is multiplied by F, where F is the product of all the stock's factors with later ex-dates. The adjusted series feed every calculation; the archived files are never changed.

Volume ratio

```
\text{rel\_vol}_{s,t} = \frac{\text{volume}_{s,t}}{\text{mean of volume}_{s}
\text{ over the 20 sessions before } t}
```

N high = valid stocks with rel_vol ⩾ 1.5; N low = valid stocks with rel_vol < 0.5; Volume Ratio = N high ÷ N low. When N low is 0 the ratio is stored empty (null) and the counts are shown instead. Each stock's 20 sessions are its own trading days, so a holiday or a suspension day simply isn't counted.

% above EMA

Each stock's EMA uses the split-adjusted close with span n (10, 20 or 50) and alpha = 2 ÷ (n + 1), seeded with the first close in a 320-session window that ends on the date (the same recursion as TradingView's `ta.ema`). % above = stocks whose close is above the EMA ÷ valid stocks × 100.

Market situation

The classifier runs for each date, bucket and EMA length, using VR and P (the % above that EMA). Rules are checked in this order:

1. P missing (no valid stocks) → no state.
2. Panic override: VR ≥ 2.0 and P < 30 → Panic.
3. Euphoria override: P ≥ 80 → Euphoria.
4. Otherwise the decision table below, by P zone and VR zone. An undefined VR counts as high.

P zone	VR low (< 0.5)	VR mid (0.5–1.0)	VR high (≥ 1.0)
VH (≥ 80)	EU	EU	EU
H (60–80)	LD	BD	BD
M (40–60)	LD	LD, or BD if VR ≥ 0.7 and dP > 0	BD
L (20–40)	LS if dP ≥ 0, else BR	BR	BR
VL (< 20)	LS	BR	PA

dP = P today − P five sessions earlier for the same bucket; in the first five sessions it is taken as 0. Values exactly on an edge fall in the upper zone (P = 40 is M; VR = 1.0 is high). Each label stores the rule that produced it (for example `table:Lxlow:alt` or `override:panic`), which the explain popovers spell out.

Smoothing. The raw state is the table's answer for that day. The confirmed state, shown by default, changes only after a new raw state has appeared on 2 consecutive sessions (`confirm_days`). A one-day blip therefore never changes the confirmed state. `days_in_state` counts consecutive sessions of the confirmed state.

What is stored per day

One row per date and bucket (5 buckets + ALL) in `data/breadth/breadth_history.csv`, readable in Excel: date, bucket, universe version, n_universe, n_valid, n_excluded, n_high, n_low, vol_ratio, vol_regime, n_above and pct for each EMA, confirmed and raw states, rule, dP and days in state for each EMA, data source and data quality. A parquet copy sits beside it, and `breadth_latest.json` holds the latest day.

Errors, fallbacks and what the UI shows

Every data fetch has a backup plan, and the app always tells you which backup was used. The table lists each failure, what the app does about it, and what you see.

Failure	What the app does	What you see
NSE refuses a request (403, 401, empty or broken reply)	Starts a fresh browser-like session, re-visits nseindia.com for cookies, retries up to 4 times with growing pauses; then tries the Playwright browser if installed.	Day marked **recovered**; yellow badge “Recovered” and a banner with the recovery details.
Bhavcopy missing on a trading day	NSE historical API per stock, then yfinance for stocks still missing; the next daily run retries the bhavcopy and replaces the day (old file kept).	**fallback**: banner “Day built from fallback source …” with the affected symbols; hollow dots on charts.
Some stocks still missing after fallbacks	Leaves them out of that day’s counts and retries on later runs.	**partial**: orange badge; counts read “valid / universe”; banner lists the symbols.
Bhavcopy for today not published yet (before 9 PM IST)	Waits; tries again on the next run.	Nothing changes until it arrives.
No bhavcopy on a holiday or weekend	Records the date as “no session”, not an error.	Nothing; the date simply has no row.

Failure	What the app does	What you see
NSE holiday calendar unavailable	Uses the cached calendar; with no cache, assumes weekdays trade and treats a missing file as “no session”.	Info banner naming the calendar source.
Index CSV download fails	Uses the last good universe version, then the seed file `universe_seed_750.csv`.	Banner with the universe version, its source and age.
Share counts unavailable for nanocap	NSE first; if NSE blocks it (it currently does), yfinance; if both fail, reuses the previous ranking.	Yellow “Universe stale” badge and “Nanocap list is stale” banner.
Network fully down	Stops after the first unreachable day and keeps all stored results.	Red “Offline / stale” badge and “Offline: showing data as of <date>” banner.
Python job crashes	Writes the error and a traceback summary to the status file; earlier data stays intact.	Red “Job failed” badge, banner with the error and a View log link.
History CSV missing or corrupt	Reads the parquet copy; failing that, the latest-day file.	Warning banner naming the file used.
Unexplained price jump (close below 0.6× or above 1.6× the previous close)	Excludes that stock for that day.	Warning banner for the latest day; full list on Data & Quality with a suggested split-list line.
Invalid row in the split list	Skips the row.	Warning banner; reason on Data & Quality.
Web server cannot start Python	Shows the error instead of running.	The job strip and banner show “could not start python -m mbi”; set `MBI_PYTHON` if Python is not on the PATH.

The UI never crashes on missing or empty data: an empty history shows an empty state with the command to run, and a single day of data still draws every chart.

Common tasks and troubleshooting

Read today's market in one minute

1. Check the header badge is green and the Session date is the latest trading day.
2. Read the Market situation card for ALL, then click through the bucket row to see where the breadth is.
3. Look at the state map trail to see which way the market is moving.
4. Glance at the timeline to see how long the current state has lasted across buckets.

Add a split or bonus

1. Open Data & Quality and copy the suggested line for the event, or write your own (Configuration files reference).
2. Check the factor and the ex-date against the NSE corporate-actions notice.
3. Add it to `data/config/split_adjustments.json` and run `python -m mbi recompute` (or press **Recompute from archive**).
4. The event disappears from the suspected list once the adjustment matches.

Fix the three seed split dates. In `split_adjustments.json`, set TDPOWERSYS to 2026-08-24, KIRLPNU to 2026-08-18 and E2E to 2026-06-05, after checking them with NSE, then recompute.

Calibrate the market situations

1. Open Config, change one threshold, and watch the preview count.
2. Compare the state timeline with periods you remember (a rally, a correction).
3. Press **Save & reclassify history**. Undo at any time with **Reset to defaults**.

Export for Excel or backtesting

- Indicator history: Data & Quality → `breadth_history.csv`, or open `data/breadth/breadth_history.csv` directly.
- One chart's data: the **CSV** button on that chart.
- One stock's prices: Stocks → **OHLCV**, or Data & Quality → symbol → Raw OHLCV / Split-adjusted.
- Every price in one file: `python -m mbi consolidate` → `data/prices/prices_all.parquet`.

Troubleshooting

Symptom	Likely cause	Fix
Badge says “No data yet”	No history has been computed	Run `python -m mbi backfill --from 2025-01-01`.
Refresh shows “could not start python”	The web server can’t find Python	Set `MBI_PYTHON` to the full path of python.exe and restart `npm start`.
Session date is not today after 6:30 PM	NSE has not published the bhavcopy yet	Refresh again later; before 9 PM IST a missing file for today is treated as “not yet published”.
“Universe stale” banner	Nanocap ranking failed or reused	Run `python -m mbi universe --refresh --nanocap-only`.
A stock’s EMA flags look wrong after a corporate action	Split or bonus not in the list	Add it (above) and recompute.
Nanocap shows fewer valid stocks than 250	Recent listings need 50 sessions of history	Expected; they join automatically.
Charts look flat with one huge spike	Linear scale on the volume ratio	Use the **Log** switch (the default).
UI shows old numbers after editing a config file	The history wasn’t recomputed	Run the apply command for that file (Configuration files reference).